Adaptive RDMA Optimization for Distributed LLM Workloads

Week 2: Implementation Plan

Nathan Gopee
MS Computer Science, SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

January 2026

1 Problem Statement

Distributed large language model inference suffers from communication bottlenecks that manifest as Time-To-First-Token (TTFT) latency. Current systems like vLLM use NCCL for GPU-to-GPU communication, which employs static routing strategies optimized for bulk transfers. This creates **head-of-line blocking**: small, latency-sensitive tensors (KV cache pages, attention scores) wait behind large bulk transfers (model weights, gradient accumulations).

This thesis develops an **adaptive RDMA middleware** that classifies tensor access patterns at runtime and routes transfers through dedicated paths—low-latency channels for frequently-accessed "hot" data and throughput-optimized channels for "cold" bulk transfers.

2 What vLLM Already Provides

Before defining contributions, it is essential to understand vLLM's existing architecture:

Feature	**vLLM Implementation**
KV Cache Management	PagedAttention with block-based allocation
Request Scheduling	Continuous batching with FCFS/priority policies
Single-Node Multi-GPU	`MultiProcExecutor` with tensor parallelism
Multi-Node Distribution	Ray + pipeline parallelism
GPU Communication	NCCL (all-reduce, all-gather)
KV Transfer (Disagg. P/D)	Connector abstraction (LMCache, SharedStorage)
Optimizations	Chunked prefill, prefix caching, speculative decoding

Table 1: vLLM capabilities this work builds upon, not replaces.

vLLM's distributed execution uses `MultiProcExecutor`, which spawns worker processes coordinated via shared-memory message queues. Inter-GPU communication occurs through NCCL, which discovers network topology at initialization but does not adapt routing based on runtime access patterns.

3 Thesis Contribution

This work introduces three components that integrate with vLLM's existing infrastructure:

3.1 Unified Telemetry Layer

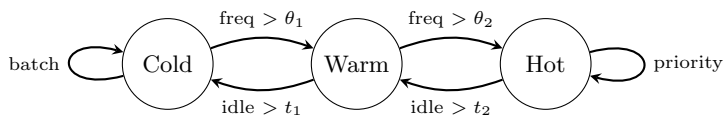
A monitoring system that instruments tensor transfers to collect:

- Access frequency per tensor/KV block
- Transfer size distributions

- Queue depth at RDMA endpoints
- Latency percentiles (p50, p95, p99)

3.2 WFA-Based Adaptive Classifier

A Weighted Finite Automaton that classifies tensors into hot/cold categories based on observed patterns. States transition based on access frequency:



The classifier also detects workload phase (prefill vs. decode, training vs. inference) to adjust thresholds dynamically.

3.3 Smart RDMA Routing Layer

Dual-path routing implemented via `pyverbs` (libibverbs Python bindings):

- **Hot Path**: Dedicated Queue Pairs (QPs) with polled completions, RDMA Write with Immediate for sub-5μs transfers
- **Cold Path**: Batched transfers with event-driven completions, optimized for throughput over latency

Memory registration uses NUMA-aware allocation via `libnuma`:

- `numa_bind_to_node()`: Pin RDMA buffers to the same NUMA node as the HCA, avoiding QPI/UPI traversal penalties (40%+ throughput loss)
- `numa_interleave_nodes()`: For large tensors spanning multiple nodes, interleave allocation across NUMA domains
- Automatic NUMA topology detection via `/sys/devices/system/node/`

4 Architecture

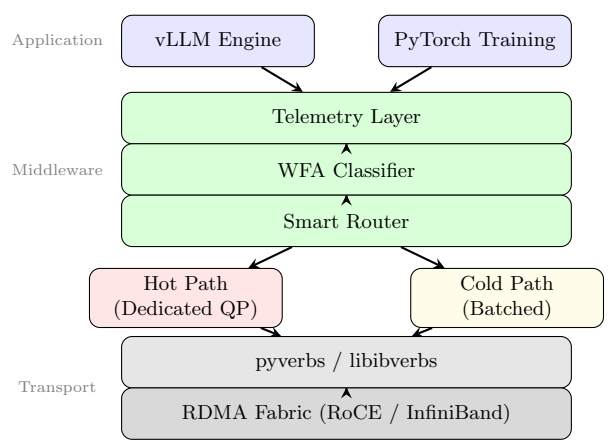


Figure 1: System architecture showing middleware integration points.

4.1 Integration Points

The middleware intercepts communication at two levels:

1. **Tensor Parallel Communication** (replacing NCCL):

```
# In vllm/distributed/parallel_state.py
def tensor_model_parallel_all_reduce(tensor):
    # Original: torch.distributed.all_reduce(tensor)
    # Modified: route through adaptive middleware
    pattern = wfa_classifier.classify(tensor)
```

```
    if pattern == "hot":
        rdma_hot_channel.all_reduce(tensor)
    else:
        rdma_cold_channel.batch_all_reduce(tensor)
```

2. KV Cache Transfers (extending vLLM's Connector abstraction):

```
# New connector: AdaptiveRDMAConnector
class AdaptiveRDMAConnector(KVConnectorBase):
    def save_kv_layer(self, layer_id, kv_tensors):
        for block in kv_tensors:
            pattern = self.classifier.classify(block)
            self.router.transfer(block, pattern)
```

5 Implementation Plan

5.1 Phase 1: Foundation (Weeks 1–4)

Step 1.1: Environment Setup

```
# Install RDMA stack on both nodes
sudo apt install rdma-core ibverbs-utils perftest libibverbs-dev
pip install pyverbs

# Enable SoftRoCE on 10GbE interface
sudo rdma link add rxe0 type rxe netdev eth0
rdma link  # Verify: rxe0 should appear

# Test connectivity
ibv_devices  # List RDMA devices
rping -s -v  # Server
rping -c -a <server_ip> -v  # Client
```

Step 1.2: Baseline Latency Measurement

```
# RDMA write latency (target: <10us for SoftRoCE)
ib_write_lat -d rxe0 -n 1000  # Server
ib_write_lat -d rxe0 -n 1000 <server_ip>  # Client

# TCP baseline for comparison
sockperf ping-pong -i <server_ip> --tcp
```

Step 1.3: pyverbs Buffer Pool Implementation

```
import pyverbs as pv
from pyverbs.pd import PD
from pyverbs.mr import MR
import numa

class RDMABufferPool:
    def __init__(self, ctx, size_mb=256, numa_node=0):
        self.pd = PD(ctx)
        # NUMA-aware allocation
        numa.bind(numa_node)
        self.mr = MR(self.pd, size_mb * 1024 * 1024,
                     pv.enums.IBV_ACCESS_LOCAL_WRITE |
                     pv.enums.IBV_ACCESS_REMOTE_WRITE)

    def get_buffer(self, size):
        # Suballocate from registered region
        ...
```

Deliverable: RDMA ping-pong achieving <10μs RTT on SoftRoCE.

5.2 Phase 2: Telemetry & Classification (Weeks 5–8)

Step 2.1: Instrument vLLM's KV Cache Manager

```
# In vllm/core/kv_cache_manager.py
class InstrumentedKVCacheManager(KVCacheManager):
    def __init__(self, ...):
        super().__init__(...)
        self.access_log = defaultdict(list)  # block_id -> timestamps

    def allocate_slots(self, request, num_tokens):
        blocks = super().allocate_slots(request, num_tokens)
        for block in blocks:
            self.access_log[block.id].append(time.monotonic_ns())
        return blocks
```

Step 2.2: Implement WFA Classifier

```
class TensorClassifier:
    COLD, WARM, HOT = 0, 1, 2
```

```
    def __init__(self, theta1=10, theta2=100, t_idle=0.1):
        self.state = {}  # tensor_id -> (state, last_access)
        self.theta1, self.theta2 = theta1, theta2
        self.t_idle = t_idle

    def classify(self, tensor_id, access_count, elapsed):
        current = self.state.get(tensor_id, (self.COLD, 0))
        if elapsed > self.t_idle:
            new_state = max(self.COLD, current[0] - 1)
        elif access_count > self.theta2:
            new_state = self.HOT
        elif access_count > self.theta1:
            new_state = self.WARM
        else:
            new_state = current[0]
        self.state[tensor_id] = (new_state, time.monotonic())
        return new_state
```

Step 2.3: Phase Detection (Prefill vs Decode)

```
def detect_phase(input_batch):
    # Prefill: many tokens, few sequences
    # Decode: few tokens (1 per seq), many sequences
    tokens_per_seq = input_batch.num_tokens / input_batch.num_seqs
    return "prefill" if tokens_per_seq > 8 else "decode"
```

Deliverable: Classifier achieving >90% accuracy on synthetic access patterns.

5.3 Phase 3: Routing Integration (Weeks 9–12)

Step 3.1: Dual-Path Queue Pair Pool

```
class DualPathRDMA:
    def __init__(self, ctx, num_hot_qps=4, num_cold_qps=2):
        # Hot path: dedicated QPs, polled completions
        self.hot_qps = [self._create_qp(ctx, poll=True)
                        for _ in range(num_hot_qps)]
        # Cold path: shared QPs, event-driven completions
        self.cold_qps = [self._create_qp(ctx, poll=False)
                         for _ in range(num_cold_qps)]
        self.cold_batch = []  # Accumulate for batching

    def transfer(self, tensor, classification):
        if classification == TensorClassifier.HOT:
            qp = self._get_least_loaded(self.hot_qps)
            self._rdma_write_immediate(qp, tensor)  # Sub-5us
        else:
            self.cold_batch.append(tensor)
            if len(self.cold_batch) >= 8:
                self._flush_cold_batch()
```

Step 3.2: vLLM Connector Integration

```
# New file: vllm/distributed/kv_transfer/adaptive_rdma.py
class AdaptiveRDMAConnector(KVConnectorBase):
    def __init__(self, config):
        self.rdma = DualPathRDMA(...)
        self.classifier = TensorClassifier()

    def save_kv_layer(self, layer_id, kv_tensors):
        for block_id, kv in enumerate(kv_tensors):
            cls = self.classifier.classify(
                f"{layer_id}:{block_id}",
                self.access_counts[block_id],
                time.monotonic() - self.last_access[block_id])
            self.rdma.transfer(kv, cls)
```

Step 3.3: Benchmark TTFT Improvement

```
# Run vLLM benchmark with adaptive connector
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-2-7b-hf \
    --tensor-parallel-size 2 \
    --kv-connector AdaptiveRDMAConnector

# Measure TTFT
python benchmarks/benchmark_serving.py \
    --model llama-7b --num-prompts 100
```

Deliverable: 30%+ TTFT reduction on 7B model inference.

5.4 Phase 4: Training Extension (Weeks 13–16)

Step 4.1: Adaptive All-Reduce for Gradients

```python
# Hook into PyTorch DDP gradient synchronization
class AdaptiveAllReduce:
    def __init__(self, process_group, rdma_pool):
        self.pg = process_group
        self.rdma = rdma_pool
        self.grad_classifier = TensorClassifier(theta1=5, theta2=50)

    def all_reduce(self, tensor):
        # Classify based on gradient magnitude variance
        cls = self._classify_gradient(tensor)
        if cls == TensorClassifier.HOT:
            # Critical gradients: immediate sync
            self.rdma.hot_all_reduce(tensor)
        else:
            # Bulk gradients: batched async
            self.rdma.cold_all_reduce_async(tensor)
```

Step 4.2: Integration with FSDP

```python
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

model = FSDP(model,
    process_group=AdaptiveProcessGroup(rdma_pool))
```

Step 4.3: Profile with Nsight Systems

```bash
nsys profile -o training_profile \
    python train.py --model llama-7b --use-adaptive-rdma
nsys stats training_profile.qdrep
```

Deliverable: Training throughput improvement of 15%+ over NCCL baseline.

5.5 Phase 5: Validation & Writing (Weeks 17–24)

Step 5.1: Comprehensive Benchmarking

```bash
# Test across model sizes
for model in llama-7b llama-13b llama-70b; do
    for tp in 2 4 8; do
        python benchmark.py --model $model --tp $tp \
            --connector {nccl,adaptive_rdma} \
            --metrics ttft,tpot,throughput
    done
done
```

Step 5.2: Ablation Studies

- WFA classifier vs. static hot/cold assignment
- Hot-path only vs. dual-path routing
- NUMA-aware vs. NUMA-oblivious allocation
- Threshold sensitivity analysis (θ_1, θ_2, t_{idle})

Step 5.3: Thesis Writing Timeline

- Week 17–18: Introduction, Background, Related Work
- Week 19–20: Design and Implementation chapters
- Week 21–22: Evaluation chapter with all figures
- Week 23–24: Conclusion, revisions, defense prep

Deliverable: Complete thesis with reproducible benchmarks and defense.

6 Expected Results

Metric	Baseline	Target	Measurement
TTFT (7B model)	∼150ms	<100ms	vLLM benchmark suite
TTFT (70B model, 2-node)	∼800ms	<500ms	vLLM benchmark suite
P99 latency variance	High (HOL)	Low	Custom telemetry
Gradient sync overhead	30% step time	<15%	PyTorch profiler

Table 2: Target performance improvements.

7 Hardware & Software Requirements

Lab Hardware:

- Cerberus: 2× RTX 5090 (32GB each)
- Chimera: 3× RTX 3090 (24GB each)
- Network: 10GbE with SoftRoCE, potential 100GbE upgrade

Software Stack:

- `vllm` (fork with connector modifications)
- `pyverbs` from `rdma-core`
- `ray` for distributed coordination
- `libnuma` for NUMA-aware allocation
- NVIDIA driver 550+, CUDA 12.4+

Key Repositories:

- `https://github.com/vllm-project/vllm`
- `https://github.com/linux-rdma/rdma-core`
- `https://github.com/NVIDIA/nccl` (baseline comparison)
- `https://github.com/linux-rdma/perftest`
- `https://github.com/teslamotors/ttpoe` – Tesla's RDMA implementation (xAI cluster optimizations)
- `https://github.com/ndg8743/x99-udp4-rebar` – Custom BIOS for Resizable BAR on X99

RDMA & InfiniBand Documentation:

- Arch Wiki: InfiniBand – Comprehensive setup guide

vLLM & Inference Framework Resources:

- Anatomy of vLLM – Official deep-dive into vLLM internals
- GPUDirect RDMA vs Standard RDMA Benchmarking – Pipeline parallel performance analysis
- vLLM vs TensorRT-LLM Performance Discussion – Community benchmarks
- Jake's Exo Distributed Inference Demo – Multi-node inference demonstration

8 Risk Mitigation

Risk	Mitigation
SoftRoCE performance insufficient	Focus on algorithmic contribution; note hardware RDMA would improve results
vLLM API changes break integration	Pin to specific commit; abstract integration layer
Classification overhead exceeds benefit	Implement lightweight WFA; batch classification decisions
Limited 100GbE access	Demonstrate on 10GbE; project improvements

9 Conclusion

This thesis addresses a concrete performance bottleneck in distributed LLM systems by introducing runtime-adaptive communication optimization. The work builds on vLLM's production-grade infrastructure while contributing a novel pattern-aware routing layer. The approach is validated on consumer hardware, demonstrating that intelligent communication scheduling can achieve latency improvements previously requiring expensive InfiniBand deployments.